Spring Boot

What's new, and how to get there from 3.x

The first major release since 2022 brings smaller modules, Jackson 3, JSpecify null safety and built-in API versioning. Most of the upgrade is renaming. Watch out for the few changes that compile fine but behave differently.

November 2025
Boot 4.0.0
Released 20 November, a week after Framework 7.0, alongside Data 2025.1 and Cloud 2025.1.[3,5]

January 2026
Cloud 2025.1.1
Needed for Boot 4.0.1 and later. 2025.1.0 only works with 4.0.0.[11]

June 2026
Boot 4.1.0[13]

June 2026
Last free 3.5 release
3.5.16 ends open source support for 3.x.[15]

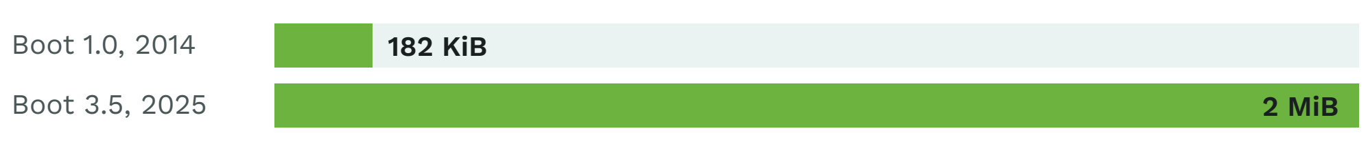
November 2026
Boot 4.2 RC[14]
General availability pending.

What's new

The changes you will touch in daily work, with code you can copy.

One module per technology

The old auto-configuration jar had grown to 2 MiB, and every class and property in it showed up in your IDE. Now each technology gets its own module, starter and test starter, all named the same way.[1,4]



Module	`spring-boot-flyway`
Starter	`spring-boot-starter-flyway`
Test starter	`spring-boot-starter-webmvc-test`
Package	`org.springframework.boot.flyway`

{ } Jackson 3 by default

New group ID and package, immutable mappers, unchecked exceptions, and support for `java.time` without an extra module.[1]

```
// Spring Boot 3.x
import com.fasterxml.jackson.databind.ObjectMapper;
// Spring Boot 4
import tools.jackson.databind.json.JsonMapper;
import com.fasterxml.jackson.annotation.JsonProperty; // unchanged
```

- Boot configures a `JsonMapper`. To replace it, define a `JsonMapper` bean, not an `ObjectMapper`.
- Every Jackson module on the classpath is registered automatically.
- Not ready to switch? The deprecated `spring-boot-jackson2` module keeps Jackson 2 running for now.

API versioning

Map one path to several versions. The version can come from a header, a query parameter, the path or the media type.[7]

```
@GetMapping(path = "/orders", version = "1.0")
OrderV1 orderV1(...)

@GetMapping(path = "/orders", version = "1.1+")
OrderV2 orderV2(...)
```

```
spring.mvc.apiversion.use.header=X-Version
spring.mvc.apiversion.default=1.0.0
```

"1.1+" matches 1.1 and every later version. WebFlux uses `spring.webflux.apiversion.*`. RestClient, WebClient and the test clients can send versions too.

HTTP service clients

Write an interface and Spring creates the client. Group clients and configure each group with properties.[8]

```
@HttpExchange("/orders")
interface OrderClient {
  @GetExchange("/{id}")
  Order get(@PathVariable String id);
}

@ImportHttpServices(group = "orders",
                    types = OrderClient.class)
```

```
spring.http.serviceclient:
  orders:
    base-url: https://orders.example.com
    read-timeout: 2s
```

Retry and concurrency limits

Retries and throttling are now part of Spring Framework 7. For most apps, that replaces Spring Retry.[6]

```
@Configuration
@EnableResilientMethods
class ResilienceConfig {}

@Retryable(includes = TimeoutException.class,
    maxRetries = 4, delay = 100, multiplier = 2)
public Quote fetchQuote() { ... }

@ConcurrencyLimit(10)
public void callLegacySystem() { ... }
```

Import from `org.springframework.resilience.annotation`, not Spring Retry's package of the same name. For programmatic retries, use `RetryTemplate` with a `RetryPolicy`. Spring Kafka and Spring AMQP retry the same way now.

Null safety

All Spring projects now declare nullness with JSpecify. Spring's own `@Nullable` and `@NonNullApi` are deprecated.[1,5]

```
// package-info.java
@NullMarked
package com.example.orders;
```

Switch to `org.jspecify.annotations.Nullable`. Kotlin reads these annotations, so some Kotlin code may stop compiling.

OpenTelemetry starter

`spring-boot-starter-opentelemetry` sets up the OpenTelemetry SDK and exports metrics and traces over OTLP.[2,17]

Micrometer has its own modules now, so you can record metrics without pulling in the full Actuator.

Testing

`RestTestClient` is a blocking counterpart to `WebTestClient`. It works against MockMvc or a running server.[2]

JUnit 6 is the managed version. Cached test contexts that sit idle are paused and restarted when a test needs them again.[5]

Spring Security 7

Adds multi-factor authentication. Spring Authorization Server is now part of Spring Security.[9]

`and()` is gone, so configure with lambdas only. `authorizeRequests` and `AntPathRequestMatcher` give way to `authorizeHttpRequests` and `PathPatternRequestMatcher`. The OAuth2 password grant is removed.

Also in 4.0

- `JmsClient`, a fluent JMS API like `JdbcClient`[2]
- Clients built on the JDK `HttpClient` use virtual threads once you enable them[2]
- Spring Data repositories generated ahead of time for faster startup[10]
- Several `TaskDecorator` beans are combined automatically[2]
- Kotlin serialization starter for JSON[2]
- A `logging.console.enabled` switch to turn console logging off[2]

Migrating from 3.x

Work in this order. You can build and test after every step.[1]

1. **Get to the latest 3.5.x** Fix every deprecation warning. Anything deprecated in 3.x is gone in 4.0.
2. **Check your dependencies** Spring Cloud needs 2025.1 (Oakwood). Confirm every other library supports Boot 4.
3. **Switch to Boot 4** Add `spring-boot-properties-migrator` at runtime. It logs renamed properties and maps them until you fix them.
4. **Compile with classic starters** `spring-boot-starter-classic` and `-test-classic` pull in every module. Fix imports and Jackson.
5. **Move to focused starters** Swap the classic starters for one starter and one test starter per technology.
6. **Test for changed behavior** Run the full suite, work through the dark box below, then drop the migrator.

OpenRewrite automates much of steps 1 to 5 with the recipe `org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0`. Review the result like any other pull request.[16]

Rename map

The renames most projects hit. For properties, the migrator reports any others.[1]

Starters

`spring-boot-starter-web`	→	`spring-boot-starter-webmvc`
`spring-boot-starter-web-services`	→	`spring-boot-starter-webservices`
`spring-boot-starter-aop`	→	`spring-boot-starter-aspectj`
`spring-boot-starter-oauth2-client`	→	`spring-boot-starter-security-oauth2-client` Same pattern for resource-server and authorization-server

Annotations and classes

`@MockBean, @SpyBean`	→	`@MockitoBean, @MockitoSpyBean`
`Jackson2ObjectMapperBuilderCustomizer`	→	`JsonMapperBuilderCustomizer`
`@JsonComponent, @JsonMixin`	→	`@JacksonComponent, @JacksonMixin`
`HttpMessageConverters bean`	→	`Server/ClientHttpMessageConvertersCustomizer`
`o.s.boot.env.EnvironmentPostProcessor`	→	`o.s.boot.EnvironmentPostProcessor`
`hibernate-jpamodelgen`	→	`hibernate-processor`

Properties

`spring.data.mongodb.uri, host, …`	→	`spring.mongodb.uri, host, …`
`spring.jackson.read.*, write.*`	→	`spring.jackson.json.read.*, json.write.*`
`spring.session.redis.*`	→	`spring.session.data.redis.*`
`spring.dao.exceptiontranslation.enabled`	→	`spring.persistence.exceptiontranslation.enabled`
`management.tracing.enabled`	→	`management.tracing.export.enabled`

Compiles fine, behaves differently

No build error will warn you. Check each one before you ship.[1]

Flyway or Liquibase migrations don't run
The library alone no longer triggers auto-configuration. Add `spring-boot-starter-flyway` or `-liquibase`.

The H2 console is gone
Add its new module, `spring-boot-h2console`.

Batch job history disappears after a restart
`spring-boot-starter-batch` keeps metadata in memory. Use `spring-boot-starter-batch-jdbc`.

MockMvc or TestRestTemplate can't be injected
`@SpringBootTest` no longer adds them. Add `@AutoConfigureMockMvc` or `@AutoConfigureTestRestTemplate`.

`@Mock` fields are null
The Mockito listener was removed. Add `@ExtendWith(MockitoExtension.class)`.

JSON reads or writes differently
Jackson 3 has new defaults. `spring.jackson.use-jackson2-defaults=true` restores the old ones while you test.

MongoDB UUIDs or decimals fail to map
There is no default representation anymore. Set `spring.mongodb.representation.uuid` and `spring.data.mongodb.representation.big-decimal`.

Health shows liveness and readiness groups
Probes are on by default. Turn them off with `management.endpoint.health.probes.enabled=false`.

Optional dependencies missing from the jar
The Maven plugin leaves them out. Set `<includeOptional>true</includeOptional>`.

Removed in 4.0

Plan a replacement before you upgrade.[1]

~~Undertow~~ use Tomcat or Jetty
~~Executable jar launch scripts~~ `java -jar` still works
~~Classic jar loader~~
~~Spock integration~~ back in 4.1
~~Reactive Pulsar client~~
~~Spring Session Hazelcast and MongoDB~~ now maintained by the vendors
~~Spring Retry version management~~ declare the version yourself

After 4.0

A new feature release ships every six months. Staying current keeps you on free support.

Spring Boot 4.1

Released 10 June 2026[13]

- Spring gRPC support
- Context propagation for `@Async`
- Address filtering against SSRF in HTTP clients
- Removes the APIs deprecated in 4.0, so clear 4.0 deprecation warnings first[12]

Spring Boot 4.2

Release candidate, November 2026[14]

- Based on Spring Framework 7.1
- `RestTemplate` becomes formally deprecated. Plan the move to `RestClient`.[5]
- OpenTelemetry semantic conventions for observations

Sources

1 Spring Boot 4.0 Migration Guide
2 Spring Boot 4.0 Release Notes
3 Spring Boot 4.0.0 available now
4 Modularizing Spring Boot
5 Spring Framework 7.0 Release Notes
6 Spring Framework reference: Resilience features
7 Spring Boot reference: Servlet web applications
8 Spring Boot reference: Calling REST services
9 What's new in Spring Security 7.0
10 Spring Data 2025.1 goes GA
11 Spring Cloud 2025.1.1 (Oakwood) released
12 Spring Boot 4.1 Release Notes
13 Spring Boot 4.1.0 available now
14 Spring Boot 4.2.0-RC2 Release Notes
15 Spring Boot 3.5.16 available now
16 OpenRewrite: Migrate to Spring Boot 4.0
17 OpenTelemetry with Spring Boot

Compiled in November 2026 from the official Spring documentation listed above. This is a community poster, not affiliated with the Spring team or Broadcom. Spring and Spring Boot are trademarks of Broadcom Inc.

Fonts: Metropolis, Work Sans, JetBrains Mono. Icons: Lucide.